

OpenMuse 持续任务与审批机制学习指南

沿着 12 个问题读真实代码，再用 PGLite 验证边界

把“任务继续跑”和“动作安全执行”拆开，用源码与实验分别验证。

对象：有工科基础、已有智能体概念储备的初学者。固定代码版本 b06caad7005ac5b6d2b451752a3794a6ae1759c1；项目为 MIT 开源应用。本指南保留 2026-10-05 固定源码与历史核心实验；本次公开整理未重跑，不是完整产品部署手册。

12 个疑问就是目录

- 01 它为什么不只是一个聊天窗口
- 02 为什么要把计划状态和结果分开存
- 03 PGLite 保存了什么 又没替你解决什么
- 04 两个 Worker 同时看到待办 谁能执行
- 05 租约过期 为什么旧 Worker 不能继续写
- 06 检查点能让程序从哪一行继续
- 07 点了取消 已发出的请求会消失吗
- 08 审批的到底是一句话 还是一份快照
- 09 重复点击和重复准备 是同一个问题吗
- 10 没收到回执 为什么不能自动再试
- 11 浏览器终端桌面 是同一种电脑能力吗
- 12 测试通过之后 到底能说自己学会了什么

历史公开知识范围：394 条索引元数据、18 篇相关全文；其余 376 篇只查元数据。不代表全库无重复，也不代表读者已掌握。

它为什么不只是一个聊天窗口

OpenMuse 把对话、长期任务、审批和结果放进同一个应用。



逐层核对职责与证据，不跳级下结论

同一件事有不同层：界面展示，服务编排，数据库保存状态，适配器才接触外部系统。

用一个具体任务理解

假设你让应用处理一份虚构的活动表单：它需要读材料、等你补充字段、保存新 PDF、提出发信审批，最后记录结果。聊天消息负责交流，任务对象负责记住这件事做到哪里；两者不是同一个数据结构。

已有知识在这里怎样落地

可复用 [[AgentHarness 智能体挽具]] 与 [[工具调用生命周期]] 的公开概念，本指南不重讲智能体基础。OpenMuse 的新增看点是：AgentTask 有 status、state、actionId、artifactIds；这些字段把等待、恢复、审批和产物关联起来。

先选一个足够小的学习切口

本次实际运行任务引擎、审批服务和 PGLite 存储的核心切片，不启动完整前后端。没有配置模型、Google、CopilotKit Intelligence 或电脑服务，因此不能把后面的测试通过读成个人助理已部署。

自测：关掉聊天页面后，任务凭什么还知道进度？

参考回答：靠服务端保存的任务状态与仍在运行的执行服务；聊天界面不是任务状态的唯一存放处。

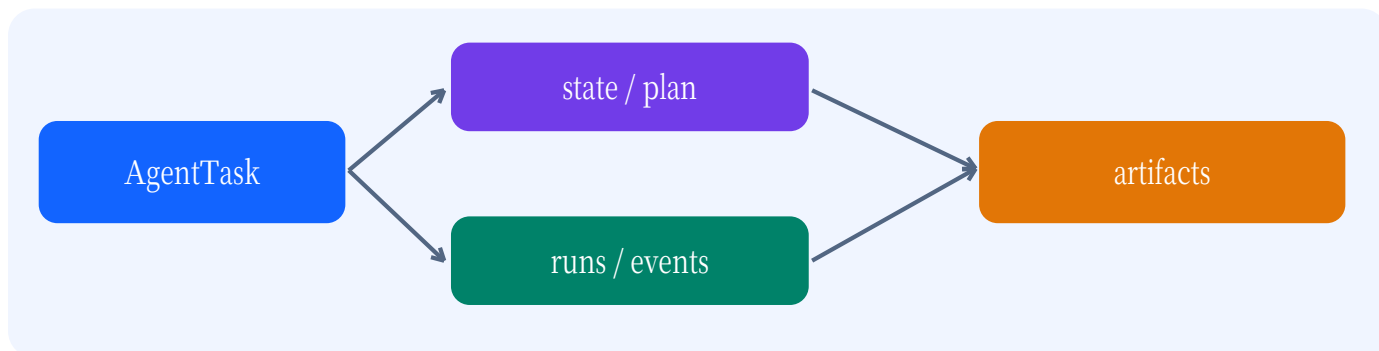
共享任务类型 packages/domain/src/agent.ts#L3-L56

服务连接任务执行器 apps/server/src/engine/service.ts#L43-L75

项目能力与前提 README.md

为什么要把计划状态和结果分开存

任务的当前状态回答“现在怎样”，运行记录回答“这次发生了什么”。



一个任务连接可变进度、历次执行记录和可展示产物。图中是数据关系，不表示每个写入都在同一事务中。

四类信息各有职责

plan 是步骤列表，state 是执行器约定的恢复数据；runs 按一次执行保存开始和结束；run-events 写可读事件；artifactIds 指向结果。把一切塞进最后一条聊天文本，程序就很难稳定判断是否该继续。

数据库怎样容纳它们

Store 使用 records 表，以 owner、kind、id 为复合主键，data 为 JSONB。JSONB 是能被 SQL 条件查询的 JSON 数据。kind 区分 tasks、actions、runs 等记录；get/list 会带 owner 条件，避免把不同拥有者的数据混在一起。

留下记录不等于完成审计

记录便于追踪，但本切片没有证明追加不可改、防篡改、完整事务审计或多租户安全认证。把它接到已有 [[产物留痕与状态外置]]：应同时看实际状态、日志与产物，不能只看一条“成功”文案。

自测：为什么一个任务可能有多个 run？

参考回答：每次领取都会建立一次执行记录；任务重试或过期接管后仍是同一任务，却是新的执行尝试。

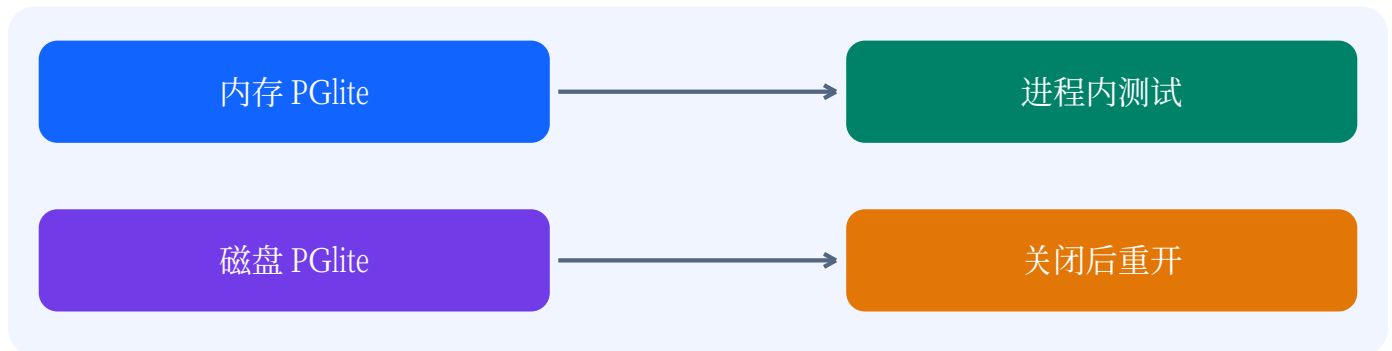
记录表与读取条件 apps/server/src/db.ts#L13-L76

记录表初始化 apps/server/src/db.ts#L122-L141

执行与事件记录 apps/server/src/engine/worker.ts#L154-L203

PGlite 保存了什么 又没替你解决什么

这次用的是真实 SQL 数据库，不是用字典假装持久化。



内存实例用于多数测试；文件目录实例用于关闭再打开的持久化检查。两条路径都执行真实 SQL。

存储选择来自代码分支

createStore 若收到 databaseUrl，就创建 PostgreSQL 连接池；否则创建 PGlite。PGlite 是嵌入式 PostgreSQL 实现。本实验使用后者；未连接独立 PostgreSQL 服务，不能宣称跨进程数据库部署已验收。

持久化的可观察证据

文件测试先写入数据，正常 close，再对同一路径 createStore 并读回。过期任务测试还会读回保存的 state。这个过程支持“正常关闭再打开后数据仍在”，而没有制造断电、进程强杀、磁盘损坏或备份恢复。

工程选择的边界

README 提醒不能由多个独立进程共同打开同一份 PGlite；独立任务进程应使用 PostgreSQL。本次两个 TaskWorker 共享一个 Store，只能观察同进程并发争抢，不能把它类推成分布式集群测试。

自测：两个 Worker 实例并发，是否已经验证两台服务器？

参考回答：没有。实例、进程、主机是不同层次，本实验只有同一进程中的多个实例。

数据库后端选择 apps/server/src/db.ts#L115-L141

磁盘关闭重开测试 tests/persistence.test.ts#L8-L22

恢复检查点测试 tests/engine.test.ts#L70-L98

部署存储边界 README.md

两个 Worker 同时看到待办 谁能执行

先读到任务还不算领取成功；必须赢得一次带条件的 SQL 更新。



两个执行器都可能读到 `queued`；`compareAndSwap` 让只有仍满足旧状态条件的一次更新返回任务。

CAS 是什么

Compare and Swap，中文可叫比较并交换：当记录仍符合 `expected` 条件时才应用 `patch`。Store 把它实现为一条 `UPDATE`，条件含 `owner`、`kind`、`id`，以及 `data` 包含指定旧字段；成功时 `RETURNING` 返回新记录，失败时返回空。

领取流程怎样利用它

`TaskWorker` 先扫描候选任务，再用旧 `status` 与 `leaseId` 作条件；若接管 `running` 任务，还比较 `leaseUntil`。胜者把任务写成 `running`，并生成新的 `leaseId`、截止时间和尝试次数。败者拿不到更新结果，立即返回。

避免过度承诺

这比“先读再无条件写”更可靠，但它只把数据库中的领取变成条件更新，不把远程发信和本地写入合并为一个原子事务。已有 [[多Agent协作乱序竞态]] 在这里有了可以指到具体 SQL 的案例。

自测：为何不能只在 JavaScript 里判断 `status` 等于 `queued`？

参考回答：读完到写入之间其他执行器可能已领取；必须在数据库更新时再次比较旧条件。

CAS SQL `apps/server/src/db.ts#L47-L58`

任务领取 `apps/server/src/engine/worker.ts#L116-L139`

同进程双 Worker 测试 `tests/engine.test.ts#L30-L45`

租约过期 为什么旧 Worker 不能继续写

leaseId 是本次执行的身份凭据；新执行接手后，旧凭据不能更新当前任务。



接管改变身份；旧执行器更新不了新任务

时间到期使任务成为可接管候选；新 leaseId 写入后，旧执行器的条件写入会失败。

租约与永久锁的区别

租约是有截止时间的执行资格。默认 leaseMs 为 60 秒，心跳约每三分之一租期尝试续约。过期 running 任务可进入候选队列，防止一个失去响应的执行器永久占住任务。

真正挡住旧写入的条件

guard 检查中止信号、当前 leaseId 和 running 状态；checkpoint 用 leaseId 与 running 做 CAS。注意 guard 本身并不单独检查墙钟是否超过 leaseUntil。到期是允许接管的条件，新租约或状态变化才使旧执行身份失配。

这道栅栏保护到哪里

栅栏在这里指“用执行身份拒绝过期执行器的写入”。这是 UUID 身份比较，不是外部提供方强制检查的单调递增 fencing token。外部请求若已经发出，数据库无法把它收回来。每次不可逆动作前仍需要守卫、审批和外部服务的结果核对。

自测：刚超过 leaseUntil，尚无人接管，guard 一定抛错吗？

参考回答：不一定。代码检查的是中止、租约身份和状态，并没有直接比较 leaseUntil 与当前时间。

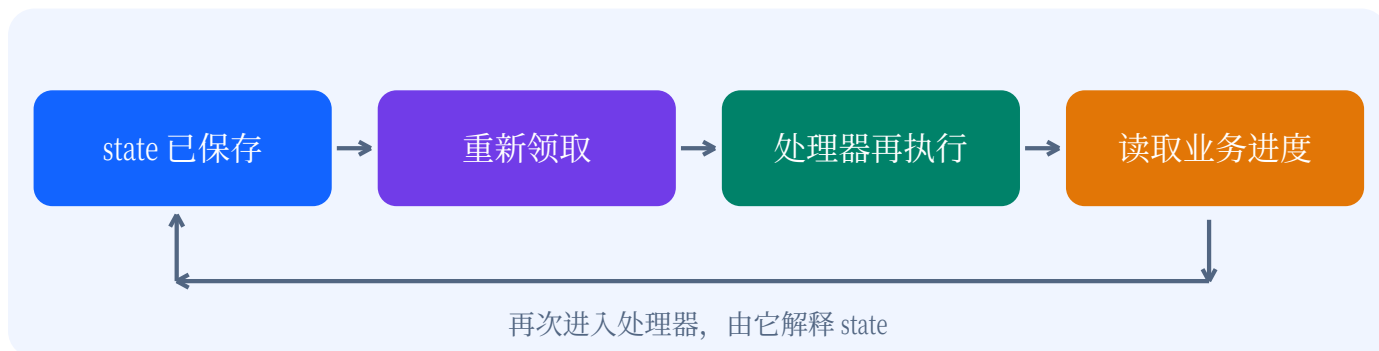
候选与过期条件 apps/server/src/engine/worker.ts#L75-L111

租约守卫和检查点 apps/server/src/engine/worker.ts#L116-L152

心跳续约 apps/server/src/engine/worker.ts#L165-L182

检查点能让程序从哪一行继续

本项目保存的是业务状态；如何跳过已完成工作，由任务处理器决定。



重新执行处理器时能拿到已存状态，不等于恢复任意 JavaScript 指令位置。

检查点里到底是什么

TaskContext.checkpoint 接收 AgentTask 的局部补丁，例如 state 中记录已完成步骤和文件 ID。它通过租约条件更新任务，再把新任务放进当前局部变量。保存粒度由调用代码决定，不是自动捕获整个进程内存。

上游测试真正观察了什么

测试先手工写一条租约已过期、state 含 completedStep 和 fileId 的记录，关闭并重开数据库。新处理器读到这份 state 并完成。测试证明状态被读回和任务可被接管，没有证明某次真实表单流程所有副作用都不会重做。

和已有 Interrupt 知识的连接

已有笔记中“恢复一定从暂停那一行继续、之前代码不重跑”过于绝对。LangGraph 的恢复也可能从节点起点重入；OpenMuse 这里更没有采用该框架的续执行承诺。需要把可重放计算和外部副作用分开，并用幂等键或结果核对保护后者。

自测：恢复时是否可以无条件重新发一遍邮件？

参考回答：不可以。先核对已保存动作状态和外部结果；检查点并不天然保证副作用只发生一次。

检查点接口与实现 apps/server/src/engine/worker.ts#L112-L122

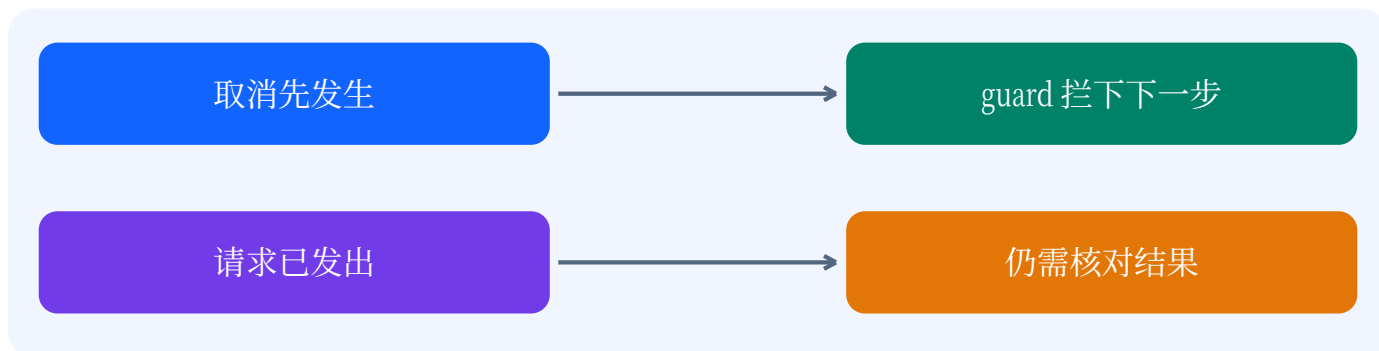
检查点条件更新 apps/server/src/engine/worker.ts#L141-L152

实际恢复测试 tests/engine.test.ts#L70-L98

LangGraph 官方恢复边界 <https://docs.langchain.com/oss/python/langgraph/interrupts>

点了取消 已发出的请求会消失吗

取消能阻止后续步骤；已经在途的外部操作仍可能完成。



时序决定边界：检查点前能拒绝新的动作，外部系统已经接收的请求不能靠本地状态撤回。

本次能够验证的取消

上游测试在处理器内把任务改成 cancelled 并清空租约，随后调用 guard，再尝试增加效果计数。guard 抛错，计数保持 0。这是“下一效果前有守卫”这一条路径的真实验证。

没有被测试替你保证的事

如果处理器没有调用 guard，或 guard 通过之后远程请求已经发出，本地取消不等于远程撤销。TaskWorker 的 AbortSignal 是协作中止信号，外部调用必须支持并正确处理它，且仍可能出现返回丢失。

等待审批也有调度含义

任务在 waiting_approval 时，如果动作仍 awaiting_review 或 executing，就继续等；动作到期可被标为 expired。测试还检查等待审批不会让排队任务饿死。等待不是执行线程一直占着一个聊天请求。

自测：界面显示 cancelled，可以直接认定钱没付或信没发吗？

参考回答：不能。要看动作状态、提供方回执及实际结果；本次完全没有调用真实支付或邮件服务。

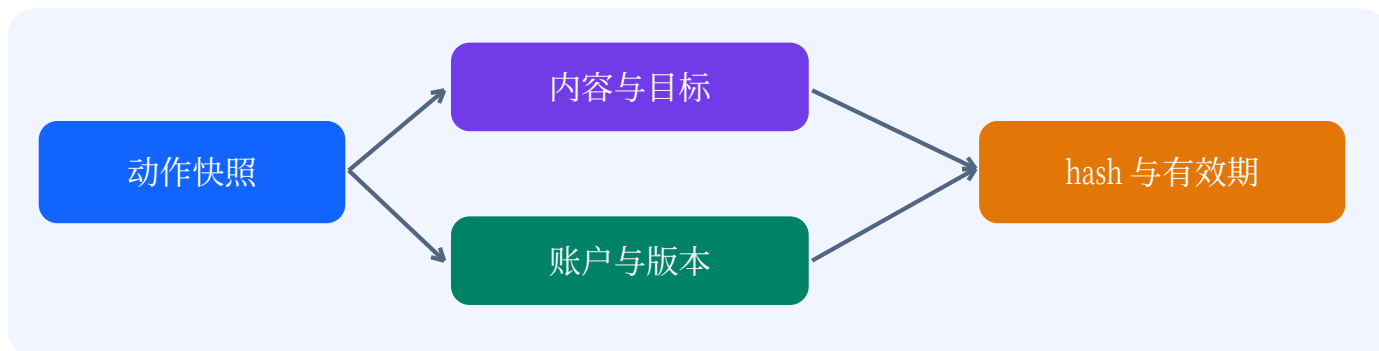
取消前的效果守卫测试 tests/engine.test.ts#L46-L68

等待审批的筛选 apps/server/src/engine/worker.ts#L85-L111

中止与丢失租约处理 apps/server/src/engine/worker.ts#L204-L241

审批的到底是一句话 还是一份快照

批准必须对应存下来的动作内容、账户连接和版本信息。



hash 把审批展示过的准备结果关联起来；它不是权限本身，也不是远程提供方的执行回执。

准备阶段做了什么

ActionService.propose 校验输入，查询连接，可调用 prepare 得到权威目标详情，再保存 proposal。hash 根据 input、connection、target、targetVersion 计算；有效期是 30 分钟。审批不是随口说一个 yes 就能替换成另一封邮件。

决定阶段再次检查什么

decide 查询对应拥有者的动作，检查 hash、状态、关联任务是否允许运行、有效期以及连接是否仍是同一账户与 connectionId。重新连接同一邮箱也可能有新的连接 ID，旧审批不能直接沿用。

版本测试容易被误读

上游测试把 revision-1 保存后，再把准备端版本改为 revision-2；批准旧提案时 fixture 仍收到旧 revision-1，且返回 succeeded。它证明保存版本被传给 execute，不证明真实 Google 服务拒绝旧 ETag；实际 HTTP 条件写校验属于另一层。

自测：目标版本变了，这个测试是否一定让旧审批失败？

参考回答：不是。这个 fixture 验证旧版本参数被传下去并返回成功；不能据此宣称远程版本冲突已实测。

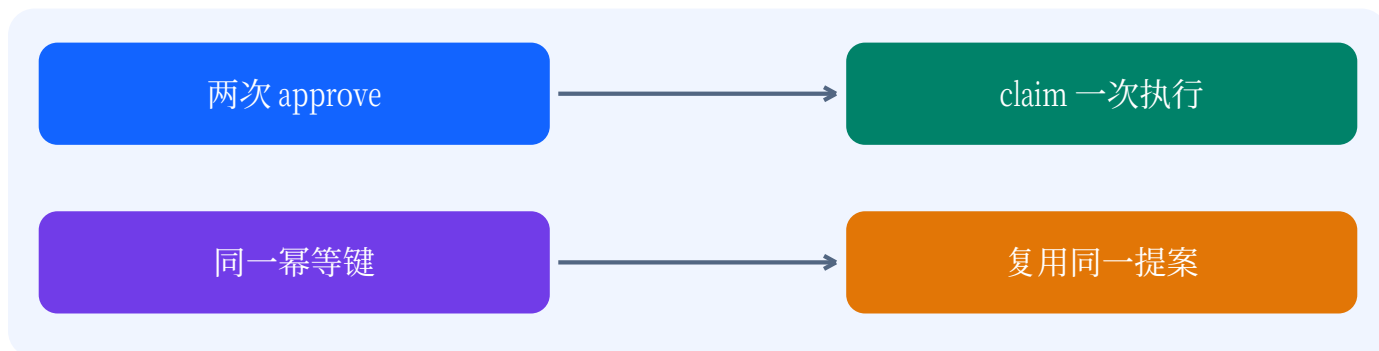
提案内容与 hash apps/server/src/actions.ts#L39-L101

决定前的校验 apps/server/src/actions.ts#L102-L149

版本 fixture 的准确语义 tests/actions.test.ts#L165-L213

重复点击和重复准备 是同一个问题吗

重复批准靠动作状态领取；重复准备靠幂等键和唯一插入。



上面保护既有动作的执行入口；下面保护准备动作时的记录身份。两条线不能互相替代。

幂等键解决哪一层

幂等性表示同一请求重复到达时，复用已有结果而不重复建立相同工作。propose 若收到 idempotencyKey，用其 SHA-256 作为动作 ID，先尝试读取已有记录，再用 insertIfAbsent 防止并发插入两个提案。主键仍包含 owner。

并发批准如何只进入一次

Store.claim 是带 awaiting_review、未过期以及关联任务状态条件的 SQL UPDATE。只有领取成功的一次 decide 才调用 execute。上游测试同时批准同一提案，检查 fixture 调用次数为 1、保存状态为 succeeded。

仍然需要设计请求身份

复用同一个幂等键会直接取回已有提案，包括已完成结果。调用者必须让键准确代表同一逻辑动作；这不是可随意复用的“万能重试按钮”。真正外部服务的一次性效果仍要考虑网络中断和提供方幂等机制。

自测：同一个键、另一个 owner，会读到前一个人的提案吗？

参考回答：本 Store 的读写主键含 owner；上游测试检查另一个拥有者得到新的待审提案。仍不等于完整多租户系统已通过安全审计。

幂等准备与唯一插入 apps/server/src/actions.ts#L39-L101

执行领取 SQL apps/server/src/db.ts#L78-L89

并发批准与复用测试 tests/actions.test.ts#L50-L68

幂等回放与竞争插入 tests/actions.test.ts#L216-L247

没收到回执 为什么不能自动再试

没有成功回执，不代表动作没有发生。outcome_unknown 要单独保留。



保留未知状态，避免盲目重复副作用

“外部是否完成”和“本地是否收到响应”是两个问题。未知状态不能轻率降级成可重试失败。

一个容易重复发信的故事

假设邮件服务已接收请求，响应却在路上丢失。如果软件把超时一律当成没发送，重试会再发一封。ActionService 识别 outcomeUnknown=true 或 code=outcome_unknown 的错误，持久化为 outcome_unknown。

本次测试观察到的行为

注入 execute fixture 首次被调用便抛出未知结果错误；再次批准同一个提案，decide 因状态已经不是 awaiting_review 而直接返回，fixture 调用次数仍是 1。没有真实邮件发出；未知情形是受控注入。

重启恢复也保持谨慎

Store.recoverInterruptedActions 可把留在 executing 的动作改为 outcome_unknown。它不是自动重放外部写入。工程上下一步应查询提供方、收集回执，再决定是否需要新动作；仅凭任务“可恢复”不足以承诺全链路 exactly-once。

自测：outcome_unknown 是否等价于 failed？

参考回答：不等价。前者承认无法确定外部是否已完成，避免盲目重试造成重复副作用。

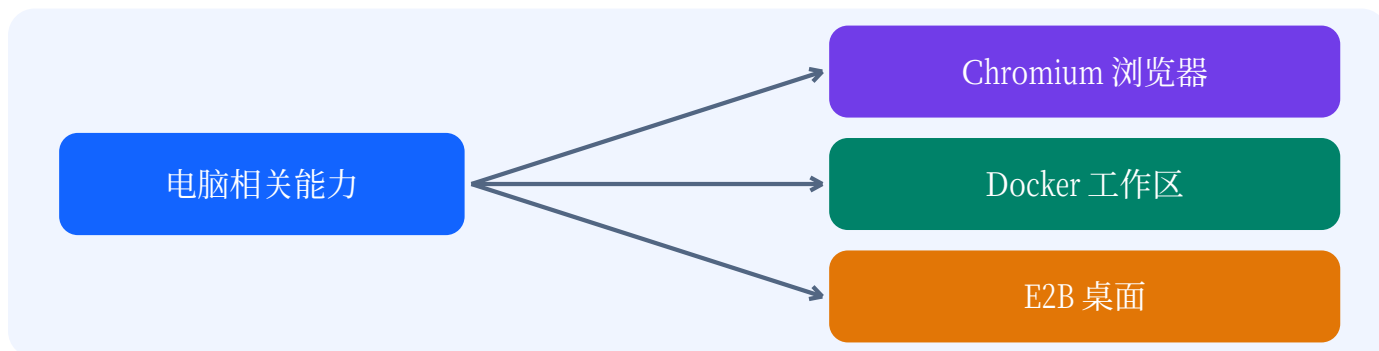
错误分类与保存 apps/server/src/actions.ts#L167-L190

未知结果不再执行测试 tests/actions.test.ts#L107-L120

执行中动作的恢复策略 apps/server/src/db.ts#L91-L95

浏览器终端桌面 是同一种电脑能力吗

当前代码已有 Docker 与 E2B 桌面分支；本次没有启动任何一种电脑。



图示区分浏览器服务与电脑后端。各自需要自己的运行环境；代码存在不等于部署完成。

接口相似 环境却不同

Browser worker 提供 Chromium 会话；ComputerBackend 提供启停、执行命令、文件等能力。DockerComputer 声明 network=disabled；配置类型还允许 e2b-desktop，E2B 分支有桌面流与 SDK 交互。不能把旧 Docker 的所有限制概括到新后端。

为什么 README 不能独自作证

固定提交的 README 仍把 graphical desktops 写成未来工作，但同一版本的 config.ts、computer.ts 与 computer-e2b-desktop.ts 已包含 E2B 桌面路径。这里以代码定位说明差异，保留“实际运行未验证”，不替上游编造统一成熟度结论。

学习时怎样划安全边界

已有 [[沙箱与审批正交]]：沙箱约束运行环境，审批约束具体动作，两者互补。本次只运行不接账号的核心测试；没有浏览器会话、E2B 密钥、Docker 容器、真实邮件或模型推理，也未接触个人电脑。

自测：能从 Docker 的 network=disabled 推出 E2B 也无网络吗？

参考回答：不能。要分别阅读和验证每个提供方的实现与部署配置。

电脑接口与 Docker 网络声明 [apps/server/src/computer-backend.ts#L12-L25](#)

Docker 具体后端 [apps/server/src/computer-backend.ts#L72-L85](#)

后端配置 [apps/server/src/config.ts#L115-L132](#)

E2B 驱动边界 [apps/server/src/computer-e2b-desktop.ts#L58-L83](#)

测试通过之后 到底能说自己学会了什么

会读一条结果的证明范围，比记住“测试全绿”更重要。



逐层核对职责与证据，不跳级下结论

结论从版本、命令、原始输出和断言推导；没有运行的层次必须保持未验证。

三种证据不要混写

代码事实：该版本确实存在某个实现。当前实跑：我们执行命令并保存原始结果。历史陈述：上游 docs/VERIFICATION.md 记录 2026-09-16 的 154 个测试和演示。第三类不能冒充 2026-10-05 在本环境完成的验收。

这个实验的真实性分层

真实层是 Store SQL、PGLite、TaskWorker 与 ActionService；受控层是注入的任务处理器、execute/prepare 提供方、模拟时钟和人为设置的状态。测试既不是纯纸面流程，也不是完整产品端到端实测。

用已有知识做最后一道检查

用 [[证据优先质检ProofOverClaims]] 检查：同进程竞争不代表跨进程；close/reopen 不代表断电；保存版本不代表 HTTP ETag 冲突已验证；通过核心测试不代表真实账号或桌面已可用。原始介绍网页未成功读取，不将其宣传作为已验证事实。

自测：怎样把“OpenMuse 没问题”改成可复核结论？

参考回答：写清固定提交、实跑命令、通过数量、真实与注入部分、失败判据和未测试范围，再指向原始日志。

历史报告日期与边界 docs/VERIFICATION.md#L1-L16

上游任务测试 tests/engine.test.ts

上游审批测试 tests/actions.test.ts

上游持久化测试 tests/persistence.test.ts

五项能力和五个具体用途

把项目作为可拆解的应用样本，先验证核心，再决定是否连接真实服务。

1 可见的任务生命周期

代码把 `queued`、`running`、等待输入、等待审批、暂停与终态存为结构化状态；可以追踪长任务进度。

2 带租约的任务领取

CAS、心跳和租约身份帮助协调执行器，允许接管保存过状态的任务；仍需处理器正确设计恢复逻辑。

3 持久审批和结果分类

动作快照、内容 `hash`、连接身份、有效期与执行领取绑定，区分拒绝、失败、成功和未知外部结果。

4 能力适配与结果展示

应用连接浏览器、电脑、文档、Google 等适配器，并提供对应结果界面；外部能力需要独立配置和验证。

5 可复用的学习切片

核心测试能在不接真实账号的条件下验证若干边界；练习包保留原始源码、许可证、版本与完整日志。

可以怎样用

1. 为自己的助理原型设计任务状态，区分任务、执行尝试、事件和最终产物
2. 给邮件或日历原型加“准备快照再批准”的流程，先用 `fixture` 验证拒绝与过期
3. 给后台任务加入租约身份检查，写一个取消后不得继续下一效果的回归测试
4. 评估开源智能体项目时，制作“源码已实现、实跑已验证、真实服务未验收”的证据清单
5. 把重复点击、版本变化、超时未知三类故障分别建模，减少盲目重试造成的重复动作

在真实 PGLite 上验证任务与审批边界

一个六步主实验：21 个选取的上游测试，加 4 个独立变式。

真实运行了什么

以下均为 2026-10-05 历史实测，本次公开整理未重跑。使用固定提交的原样ActionService、TaskWorker、Store和相关测试。SQL与PGLite是真实实现；提供方、处理器、连接状态和时钟可被注入。练习ZIP保留源码、MIT许可、依赖锁、脚本与原始日志。原始连续运行于11:46:11至11:46:23 UTC；下面六组完整命令已由独立审核者在最终ZIP新解压目录逐组复跑。两轮仍是同25个用例，不累加成50个。

环境与依赖

本环境为Linux，Node v24.19.0、npm 11.9.0、Python3、Bash。核心依赖为PGLite 0.3.16、pg 8.23.0、zod 4.6.5，含传递依赖共16项，版本及完整性与固定上游锁一致。TypeScript转换使用Node实验特性，警告保留。

怎样开始

解压练习ZIP后，在终端进入含README.md和run-core.sh的目录，再依次执行六步。命令没有安装Node或Python，缺少它们时先停止。本指南给出的是Linux/Bash实际验证路径，不声称其他平台已复验。

完成标准

成功标准：原始测试12+8+1=21通过，新变式4通过，失败0；每组退出0且NETWORK_GUARD attempts=0，最终15项源码指纹与16项依赖完整性通过。源码另有1项持久化文件测试未选择，Node日志skipped=0。

不能从中推出什么

没有完整应用启动、模型、真实Google账户、CopilotKit Intelligence、浏览器、Docker或E2B验收。网络guard是进程内辅助观察，不是操作系统隔离证明；数据库重开与虚拟时钟不是生产崩溃或分布式压力试验。

步骤 01 先核对版本与材料

```
node --version; npm --version; python3 verify-exercise.py
```

目的

确认运行时版本、原始源码复制件和依赖锁一致，先把“跑的是谁”说清楚。

实际执行时间

独立审核者从最终ZIP新目录解压后的同命令复跑：2026-10-05 11:51:11 UTC；相关命令均退出0。

实际结果

源文件核验15项，锁文件依赖16项。Node v24.19.0，npm 11.9.0。

原始输出摘录

```
SOURCE_SHA256_OK files=15  
NPM_LOCK_INTEGRITY_OK packages=16
```

验证成功的方法

两项检查均输出 OK 且最终退出码为0；verify脚本逐项比对SHA-256和上游锁的依赖完整性。

失败判据与限制

任一文件摘要或依赖完整性不一致都停止；不要略过检查继续跑。

完整 stdout/stderr、退出码及恢复过程见实验日志附件；摘录不替代原始记录。

步骤 02 只装核心所需依赖

```
bash install-deps.sh; python3 verify-exercise.py
```

目的

从官方npm仓库安装固定16项依赖，禁用生命周期脚本；不装整个应用。

实际执行时间

独立审核者从最终ZIP新目录解压后的同命令复跑：2026-10-05 11:51:26 至 11:51:33 UTC，安装约7秒；相关命令均退出0。

实际结果

安装脚本采用npm ci并读取随包lock；不加载用户npm配置，保留平台代理与证书变量，未关闭证书校验。

原始输出摘录

```
added 16 packages
SOURCE_SHA256_OK files=15
NPM_LOCK_INTEGRITY_OK packages=16
```

验证成功的方法

安装退出码为0，随后源码与锁核验通过。安装阶段需可访问npm仓库；测试阶段无需网络或账号。

失败判据与限制

网络、代理、证书或锁文件问题应保留报错并修复原因；不能用关闭TLS校验绕过。初次准备经历配置冲突、DNS与证书问题，完整记录在日志。

完整 stdout/stderr、退出码及恢复过程见实验日志附件；摘录不替代原始记录。

步骤 03 验证审批的拒绝与一次领取

```
bash run-core.sh actions
```

目的

运行原样上游actions.test.ts，覆盖拒绝、错拥有者、旧hash、到期、断连、换账号、并发批准和未知结果等。

实际执行时间

独立审核者从最终ZIP新目录解压后的同命令复跑：2026-10-05 11:51:48 至 11:51:49 UTC；相关命令均退出0。

实际结果

12个原始审批测试通过。真实ActionService和PGlite执行；execute、prepare、connected等是受控fixture。

原始输出摘录

```
# tests 12
# pass 12
# fail 0
NETWORK_GUARD attempts=0
EXIT_CODE=0
```

验证成功的方法

检查rawlogs/03-original-actions.log中的12pass、0fail、退出码0；并发批准断言fixture调用1次，拒绝类断言不调用。

失败判据与限制

任何断言失败或网络guard尝试非0都需要调查。版本测试只检查保存版本转交fixture，不代表真实HTTP If-Match或Google写入。

完整 stdout/stderr、退出码及恢复过程见实验日志附件；摘录不替代原始记录。

步骤 04 看两个执行器如何争抢与取消

```
bash run-core.sh engine; cat rawlogs/04-original-engine.log
```

目的

运行原样任务引擎测试，观察领取竞争、取消前守卫、恢复保存状态、调度和审批公平性。

实际执行时间

独立审核者从最终ZIP新目录解压后的同命令复跑：2026-10-05 11:51:58 至 11:52:04 UTC；相关命令均退出0。

实际结果

8个原始引擎测试通过。双Worker共享同一个PGlite；取消测试在效果前调用guard，效果计数保持0。

原始输出摘录

```
# tests 8
# pass 8
# fail 0
NETWORK_GUARD attempts=0
EXIT_CODE=0
```

验证成功的方法

日志应为8pass；原始测试断言一次领取、保存state读回、run开始时间，以及写运行记录失败后worker.stop能完成且任务标为failed。

失败判据与限制

这是同进程Worker对象竞争；不是多进程数据库或分布式故障验收。自定义处理器若不调用守卫，不能套用取消保护结论。

完整 stdout/stderr、退出码及恢复过程见实验日志附件；摘录不替代原始记录。

步骤 05 关闭数据库再打开

```
bash run-core.sh persistence; cat rawlogs/05-original-persistence.log
```

目的

对真实嵌套目录PGlite检查持久记录和中断动作恢复，不依赖内存字典。

实际执行时间

独立审核者从最终ZIP新目录解压后的同命令复跑：2026-10-05 11:52:16 至 11:52:18 UTC；相关命令均退出0。

实际结果

只选择名称含fresh nested data directory的1项原始测试。22项源码测试定义中，本轮选21项；另一项PostgreSQL池错误测试未选择。

原始输出摘录

```
# tests 1
# pass 1
# fail 0
# skipped 0
NETWORK_GUARD attempts=0
EXIT_CODE=0
```

验证成功的方法

run-core明确加入--test-name-pattern="fresh nested data directory"；测试close/reopen后调用recoverInterruptedActions，断言留存executing转为outcome_unknown。

失败判据与限制

Node此次报告skipped=0，不能凭排除1项改写成skipped=1。该实验正常关闭再打开，不是进程强杀、断电或崩溃耐久性测试。

完整 stdout/stderr、退出码及恢复过程见实验日志附件；摘录不替代原始记录。

步骤 06 改输入和时间 观察边界变化

```
bash run-core.sh characterization; cat rawlogs/06-changed-input-characterization.log;  
python3 verify-exercise.py
```

目的

运行独立编写的4个变式：新键改内容、同键改内容、未知结果重开、虚拟时钟触发接管。

实际执行时间

独立审核者从最终ZIP新目录解压后的同命令复跑：2026-10-05 11:52:28 至 11:52:32 UTC；相关命令均退出0。

实际结果

4个新增变式通过：新键改主题令hash变化；同键改主题返回原提案；未知结果重开后仍不重放；推进60001ms后新Worker接管，旧checkpoint和下一守卫效果被拒绝。

原始输出摘录

```
changed_subject=true changed_hash=true  
stale_hash_rejected=true wrong_owner_rejected=true provider_calls=0  
original_payload_returned=true action_count=1  
provider_calls=1 reopened_database=true  
final_status=outcome_unknown replay_calls=0  
claims=2 stale_checkpoint_rejected=true  
old_guarded_effects=0 new_handler_calls=1 final_result=new_worker  
# pass 4  
NETWORK_GUARD attempts=0  
EXIT_CODE=0
```

验证成功的方法

查看完整OBSERVATION行；检查旧checkpoint被拒绝、新处理器1次、总领取2次、最终结果new_worker，最后再次验证源码与依赖未变。

失败判据与限制

这些是刻意编写的边界刻画，不是上游新增官方测试。虚拟时钟与注入提供方属于受控输入；不能声称真实服务曾超时或真实等待60秒。

完整 stdout/stderr、退出码及恢复过程见实验日志附件；摘录不替代原始记录。

把机制接回已有概念

保留公开知识关联与版本化修正；复用前核对同义概念和已有编辑。

已有基础只补具体落点

复用智能体挽具、工具生命周期、状态外置、人机协同、并发竞态和证据优先笔记；已有笔记不等于本人已掌握。新篇幅放在具体 SQL、租约身份、审批快照和未知结果。

增量一 审批绑定

定义：批准指向确定的动作内容、连接身份、资源版本与有效期。优先增补 [[人机协同Interrupt]] 和 [[工具调用三段式流水线]]；hash 不等于签名或权限。

增量二 租约条件写入

定义：执行器写回时核对自己仍拥有当前任务。关联 [[多Agent协作乱序竞态]] 和 [[Checkpoint存档与持久执行]]；UUID 身份不是外部系统的单调 fencing token。

增量三 未知结果与幂等身份

outcome_unknown 保留外部效果的不确定性，不能盲目重发。幂等键标识一次逻辑动作，与内容 hash 用途不同；同键改内容可能直接取回旧提案。

修正一项已知误解

LangGraph 恢复会从中断节点起点重入；完整快照位于 super-step 边界，并有节点任务写入。这不等于任意代码行续跑，也不能套用为 OpenMuse 的恢复保证。官方证据见知识附件。

查重结论和复用范围

历史 394 条元数据和 18 篇已读公开正文中，OpenMuse 名称变体未命中；其余 376 篇未读正文，不代表全库无重复。复用前核对目标知识库的同义条目与已有编辑。

让结论可以被复核

固定来源

CopilotKit/openmuse, 提交 b06caad7005ac5b6d2b451752a3794a6ae1759c1。练习包保留选取源码与上游 MIT LICENSE, 清单逐文件给出 SHA-256。实际复制文件与 repo 原件做字节校验；完整上游仓库不放入 ZIP。

本次证据

实验命令、实际输出、退出码、运行时间和断言见实验日志；知识附件列出实际读过的公开笔记与查重范围。独立审核日志由另一位审核者生成，不用作者自检冒充独立复核。

历史与当前分开

上游 2026-09-16 的 154 tests、Chromium、Docker、手机演示属于历史报告；当前这次只承认日志所列核心实验。原始介绍网页未成功读取，不将其宣传作为已验证事实。

PDF 和 HTML 的检查状态

公开 PDF 使用 ReportLab 直接生成，转为逐页 PNG 后实际查看。HTML 保留内联 SVG、A4 样式与章节链接，仅结构核查；浏览器视觉及 HTML 转 PDF 原路线受阻，本次未重试。

服务与许可证边界

仓库 MIT 许可不等于第三方服务免费或一并开源。完整应用还涉及独立配置和服务条款；这次没有输入任何真实密钥，也没有连接账号、发邮件、启动模型、浏览器、Docker 或 E2B。

交付与最后一步

文件共六类：PDF、HTML、练习 ZIP、实验日志、知识笔记、独立审核日志。本次公开整理不增加新的实验或服务验收；应将历史结果与当前文件检查分开阅读。

固定版本源码入口

<https://github.com/CopilotKit/openmuse/blob/b06caad7005ac5b6d2b451752a3794a6ae1759c1/>